

Strange Blog

Cloud & Edge Computing

Alex Billa

Corso: Cloud & Edge Computing

Anno: 2025/2026

Repository: [GitHub](#) — [Strange Blog](#)

15 aprile 2026

0. Indice

1	Project Description	2
1.1	Contesto e obiettivi	2
1.2	Funzionamento dell'applicazione	2
1.3	Obiettivi DevOps	2
2	Tecnologie utilizzate	3
3	Workflow DevOps Adottato	3
4	Setup Iniziale e Configurazione	4
4.1	Configurazione dell'ambiente Python	4
4.2	Configurazione del database	4
5	Script di Validazione Markdown	4
5.1	Motivazione	4
5.2	Implementazione	5
6	Containerizzazione	5
6.1	Dockerfile	5
6.2	Entrypoint Script	6
6.3	Docker Compose	7
7	Pipeline CI/CD	8
7.1	Struttura della pipeline	8
7.2	Trigger della pipeline	10
7.3	Deployment sull'homelab con Tailscale	10
8	Problemi riscontrati e soluzioni applicate	11
8.1	Verifica dei post Markdown solo a runtime	11
8.2	Race condition tra container app e database	11
8.3	Migrazioni non applicate al riavvio	11
8.4	Server homelab non raggiungibile da GitHub Actions	11
9	Esperimenti eseguiti	11
9.1	Test dello script di validazione con post errati	11
9.2	Test della pipeline CI su Pull Request	12
9.3	Test del deploy automatico sull'homelab	12
9.4	Verifica della persistenza dei dati	12
10	Risultati	12
11	Limitazioni e possibili miglioramenti futuri	13
11.1	Limitazioni attuali	13
11.2	Possibili miglioramenti	13

1. Project Description

1.1. Contesto e obiettivi

Strange Blog è un'applicazione web sviluppata in Python con il framework Flask, concepita come blog personale e curriculum vitae online. Il progetto è stato fornito come base su cui applicare le pratiche DevOps.

L'applicazione è volutamente *subottimale dal punto di vista strutturale*: l'intera attività si è concentrata sull'automazione dei processi di sviluppo, verifica della qualità e deployment.

1.2. Funzionamento dell'applicazione

L'app si compone di due parti principali:

- **Blog posts**: i post sono file Markdown collocati nella cartella `posts/en/`. Ciascun file deve rispettare un template specifico con campi obbligatori (titolo, sottotitolo, autore, data, permalink, ecc.). L'app li legge e li renderizza tramite Flask-FlatPages.
- **Post views**: ogni accesso a un post viene registrato in una tabella `post_views` su un database PostgreSQL, gestita tramite Flask-SQLAlchemy e Flask-Migrate.

Il server applicativo utilizzato in produzione è **Gunicorn**, avviato con 4 worker sulla porta 8080.

1.3. Obiettivi DevOps

L'obiettivo dell'assignment è passare da una gestione manuale e non verificata del progetto a un workflow strutturato secondo le best practice DevOps:

1. Portare l'applicazione in un ambiente containerizzato e riproducibile
2. Automatizzare la validazione dei contenuti prima di ogni merge
3. Creare una pipeline CI/CD che copra build, test e deployment
4. Realizzare un deployment automatico

2. Tecnologie utilizzate

Tecnologia	Ruolo nel progetto
Python 3.11	Linguaggio dell'applicazione Flask
Flask 2.2	Web framework; gestisce routing e rendering dei template
Flask-Migrate	Gestione delle migrazioni del database
Gunicorn 20.1	Server WSGI per l'esecuzione in produzione
PostgreSQL 15+	Database relazionale per il conteggio delle visualizzazioni
Docker	Containerizzazione dell'applicazione e del database
Docker Compose	Orchestrazione multi-container (app + db)
GitHub Actions	Piattaforma CI/CD per l'esecuzione automatica della pipeline
Tailscale	VPN mesh per rendere raggiungibile l'homelab dalla pipeline
Bash	Scripting per la validazione dei Markdown e l'entry-point

Tabella 1: Tecnologie utilizzate nel progetto

3. Workflow DevOps Adottato

Prima di descrivere i dettagli implementativi, è utile illustrare il flusso di lavoro complessivo adottato, ispirato al **GitLab/GitHub Flow**.

1. Il maintainer apre una **Issue** su GitHub che descrive la modifica da apportare
2. Crea un branch dedicato a partire da **dev**
3. Sviluppa la modifica (nuovo post, script, configurazione)
4. Apre una **Pull Request** verso **dev** (o verso **main** per i release)
5. La pipeline CI parte automaticamente: valida i Markdown e verifica la build Docker
6. Se tutto passa, la PR viene approvata e mergiata
7. Al merge su **main**, la pipeline esegue il deploy automatico sull'homelab

I branch principali sono:

- **main** — codice in produzione; ogni merge attiva il deploy
- **dev** — integrazione continua; le feature branch confluiscono qui

4. Setup Iniziale e Configurazione

4.1. Configurazione dell'ambiente Python

Il primo passo è stato predisporre l'ambiente di sviluppo locale, seguendo le istruzioni del README fornito inizialmente.

```
1 # Creazione e attivazione del virtualenv
2 virtualenv venv
3 source venv/bin/activate
4
5 # Installazione dipendenze
6 pip install -r requirements.txt
```

Listing 1: Setup ambiente Python locale

4.2. Configurazione del database

Il file `app.py` fornito dal docente conteneva l'URL di connessione al database con IP e password hardcodati:

```
1 app.config['SQLALCHEMY_DATABASE_URI'] = \
2     "postgresql://flask:passwordhere@172.17.0.2:5432/blog"
```

Listing 2: Configurazione originale hardcodata in `app.py`

Per il test locale, è stato avviato un container PostgreSQL e l'URL è stato aggiornato con `localhost`. La sequenza di comandi per inizializzare le tabelle è:

```
1 export FLASK_APP=app.py
2 flask db init          # Crea la cartella migrations/
3 flask db migrate       # Genera lo script di migrazione
4 flask db upgrade       # Applica le migrazioni al database
```

Listing 3: Inizializzazione del database con Flask-Migrate

Dopo aver verificato il corretto funzionamento dell'applicazione in locale tramite `flask run`, si è proceduto con le attività DevOps.

5. Script di Validazione Markdown

5.1. Motivazione

Il flusso di pubblicazione prevede che il maintainer aggiunga file Markdown nella cartella `posts/en/`. Se un file contiene errori e non è conforme al template dei post (campo obbligatorio mancante, formato data errato, separatore `--` assente), l'applicazione Flask può crashare silenziosamente o produrre comportamenti imprevisti. L'errore emergerebbe solo a runtime, potenzialmente in produzione.

La soluzione DevOps corretta è intercettare questi errori *prima* che il file raggiunga il repository, tramite uno script eseguito nella pipeline CI.

5.2. Implementazione

Lo script `check_markdown_validity.sh` scorre tutti i file `.md` presenti in `posts/en/` e verifica:

- Presenza di tutti i campi obbligatori del template (9 campi)
- Formato della data: deve rispettare `%B %d, %Y` (es. *July 20, 2025*)
- Validità logica della data
- Presenza del separatore `--` tra metadati e contenuto

Lo script restituisce `exit code 1` in caso di errori, bloccando la pipeline CI.

6. Containerizzazione

6.1. Dockerfile

L'applicazione è stata containerizzata a partire dall'immagine ufficiale `python:3.11-slim`, scelta per il ridotto footprint rispetto all'immagine `python:3.11` completa.

Viene poi eseguito lo script `check_markdown_validity.sh` **durante la build dell'immagine** (riga RUN): questo garantisce che nessuna immagine Docker valida possa essere costruita se i post presenti nel repository non superano la validazione. Rappresenta un secondo livello di difesa oltre alla pipeline CI.

```
1 FROM python:3.11-slim
2
3 ENV PYTHONDONTWRITEBYTECODE 1
4 ENV PYTHONUNBUFFERED 1
5 ENV FLASK_APP app.py
6
7 WORKDIR /app
8
9 RUN apt-get update && apt-get install -y libpq-dev gcc \
10     && rm -rf /var/lib/apt/lists/*
11
12 COPY requirements.txt .
13 RUN pip install --no-cache-dir -r requirements.txt
14
15 COPY . .
16
17 RUN chmod +x /app/check_markdown_validity.sh
18 RUN /app/check_markdown_validity.sh
19
20 RUN chmod +x /app/entrypoint.sh
21
22 EXPOSE 8080
23
24 ENTRYPOINT ["/app/entrypoint.sh"]
25 CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:8080", "wsgi:app"]
```

Listing 4: Dockerfile dell'applicazione

6.2. Entrypoint Script

Un problema classico in ambienti Docker Compose è la *race condition* tra il container dell'applicazione e quello del database: il container web può tentare di connettersi al database prima che questo sia pronto ad accettare connessioni.

Lo script `entrypoint.sh` risolve questo problema con un loop di attesa attiva sulla porta TCP del database, e applica automaticamente le migrazioni prima di avviare Gunicorn:

```
1  #!/bin/bash
2  set -e
3
4  DB_HOST="db"
5  DB_PORT=5432
6
7  echo "Attesa database ($DB_HOST:$DB_PORT)..."
8
9  until timeout 1s bash -c "echo > /dev/tcp/$DB_HOST/$DB_PORT "  
10 2>/dev/null; do  
11     echo "...database non raggiungibile, riprovo tra 1s..."  
12     sleep 1  
13 done
14
15 echo "Database connesso! Applico le migrazioni..."
16 flask db upgrade
17
18 echo "Avvio Gunicorn..."
19 exec "$@"
```

Listing 5: Script `entrypoint.sh`

6.3. Docker Compose

Il file `docker-compose.yml` orchestra i due servizi (applicazione e database) su una rete bridge dedicata con subnet statica, in modo che l'IP del database sia prevedibile e configurabile in `app.py`. Il servizio `web` non esegue una build locale: scarica direttamente l'immagine precostruita e pubblicata su GitHub Container Registry dalla pipeline CI/CD:

```
1  services:
2    db:
3      image: postgres:latest
4      container_name: flask-db-test
5      environment:
6        POSTGRES_USER: flask
7        POSTGRES_PASSWORD: passwordhere
8        POSTGRES_DB: blog
9      ports:
10       - "5432:5432"
11      networks:
12        blog_network:
13          ipv4_address: 172.28.0.2
14      volumes:
15       - ./postgres_data:/var/lib/postgresql
16
17    web:
18      image: ghcr.io/alexbilla02/cloud_flask_project:latest
19      container_name: strange-blog-app
20      command: gunicorn -w 4 -b 0.0.0.0:8080 wsgi:app
21      ports:
22       - "8080:8080"
23      depends_on:
24       - db
25      networks:
26       - blog_network
27      environment:
28       - FLASK_APP=app.py
29
30  networks:
31    blog_network:
32      ipam:
33        config:
34          - subnet: 172.28.0.0/16
```

Listing 6: `docker-compose.yml`

Il volume `./postgres_data` garantisce la persistenza dei dati tra i riavvii dei container.

7. Pipeline CI/CD

7.1. Struttura della pipeline

La pipeline è definita nel file `.github/workflows/pipeline.yml` e si articola in tre job sequenziali:

1. **lint-posts**: valida tutti i file Markdown presenti in `posts/en/`
2. **build-and-push**: costruisce l'immagine Docker e la pubblica su GitHub Container Registry (GHCR); eseguito solo su `main`
3. **deploy**: scarica l'immagine aggiornata dal registry e riavvia i container sull'homelab tramite SSH (solo su `main`)

Tutti e tre i job sono sequenziali tramite la direttiva `needs`. I job **build-and-push** e **deploy** sono condizionati al branch `main` tramite `if: github.ref == 'refs/heads/main'`, quindi su `dev` e nelle Pull Request viene eseguito solo **lint-posts**.

La pipeline richiede il permesso `packages: write` per poter fare il push dell'immagine sul registry GHCR, autenticandosi con il token automatico `GITHUB_TOKEN` fornito da GitHub Actions.

```
1 name: Strange Blog Pipeline
2
3 on:
4   push:
5     branches: [ main, dev ]
6   pull_request:
7     branches: [ main ]
8
9 permissions:
10  contents: read
11  packages: write
12
13 jobs:
14   lint-posts:
15     runs-on: ubuntu-latest
16     steps:
17       - uses: actions/checkout@v4
18       - name: Permessi allo script
19         run: chmod +x check_markdown_validity.sh
20       - name: Validazione post Markdown
21         run: ./check_markdown_validity.sh
22
23   build-and-push:
24     needs: lint-posts
25     runs-on: ubuntu-latest
26     if: github.ref == 'refs/heads/main'
27     steps:
28       - uses: actions/checkout@v4
29
30       - name: Lowercase repo name
```

```
31     run: echo "IMAGE_NAME=${GITHUB_REPOSITORY,,}" >> ${
32         GITHUB_ENV}
33
34     - name: Login a GitHub Container Registry
35       uses: docker/login-action@v3
36       with:
37         registry: ghcr.io
38         username: ${ secrets.github.actor }
39         password: ${ secrets.GITHUB_TOKEN }
40
41     - name: Build and Push immagine
42       uses: docker/build-push-action@v5
43       with:
44         context: .
45         push: true
46         tags: ghcr.io/${ env.IMAGE_NAME }:latest
47
48     deploy:
49       needs: build-and-push
50       if: github.ref == 'refs/heads/main'
51       runs-on: ubuntu-latest
52       steps:
53         - uses: actions/checkout@v4
54         - name: Connessione Tailscale
55           uses: tailscale/github-action@v2
56           with:
57             authkey: ${ secrets.TS_AUTHKEY }
58         - name: Deploy via SSH
59           uses: appleboy/ssh-action@v1.0.3
60           with:
61             host: ${ secrets.SERVER_IP }
62             username: ${ secrets.SERVER_USER }
63             key: ${ secrets.SSH_PRIVATE_KEY }
64             script: |
65               cd ~/cloud/Cloud_flask_project
66               git pull origin main
67               docker compose pull
68               docker compose up -d
69               docker image prune -f
```

Listing 7: pipeline.yml — GitHub Actions

7.2. Trigger della pipeline

Evento	Branch	Job eseguiti
push	main	lint-posts, build-and-push, deploy
push	dev	lint-posts
pull_request	main (target)	lint-posts

Tabella 2: Trigger e job corrispondenti

7.3. Deployment sull'homelab con Tailscale

Il server di deployment è un homelab privato, non raggiungibile dalla rete pubblica. Per consentire a GitHub Actions di accedervi in SSH, è stato utilizzato **Tailscale**: una VPN mesh che assegna un indirizzo IP stabile e privato al server, accessibile dai nodi autorizzati della rete Tailscale.

Il processo di connessione sicura è il seguente:

1. L'action `tailscale/github-action@v2` connette il runner GitHub alla rete Tailscale usando una chiave di autenticazione monouso (`TS_AUTHKEY`)
2. Una volta connesso alla VPN, il runner raggiunge il server tramite il suo IP Tailscale
3. L'action `appleboy/ssh-action` esegue i comandi di deploy via SSH, autenticandosi con una chiave privata

Le credenziali sensibili sono gestite tramite i **GitHub Secrets**:

Secret	Contenuto
<code>TS_AUTHKEY</code>	Chiave di autenticazione Tailscale (ephemeral key)
<code>SERVER_IP</code>	Indirizzo IP del server nella rete Tailscale
<code>SERVER_USER</code>	Username SSH sul server
<code>SSH_PRIVATE_KEY</code>	Chiave privata SSH per l'accesso al server

Tabella 3: GitHub Secrets configurati per il deploy

Il comando di deploy aggiorna il codice con `git pull`, ricostruisce l'immagine e riavvia i container con `docker compose up -build -d`, quindi rimuove le immagini obsolete con `docker image prune`.

8. Problemi riscontrati e soluzioni applicate

8.1. Verifica dei post Markdown solo a runtime

Problema: Un post con formato errato (data sbagliata, campo mancante) veniva scoperto solo quando l'app crashava in produzione, senza nessun avviso preventivo al maintainer.

Soluzione: Scrittura dello script `check_markdown_validity.sh` con doppia esecuzione: nella pipeline CI (blocca la PR se ci sono errori) e nel `Dockerfile` durante la build (blocca la costruzione dell'immagine).

8.2. Race condition tra container app e database

Problema: Al primo avvio con `docker compose up`, il container dell'applicazione partiva prima che PostgreSQL fosse pronto ad accettare connessioni. Il risultato era un errore di connessione immediato e il crash del container web.

Soluzione: Implementazione dello script `entrypoint.sh` che, prima di avviare Gunicorn, esegue un loop di polling sulla porta TCP 5432 del servizio db. Solo quando la porta risponde il flusso prosegue con `flask db upgrade` e l'avvio del server.

8.3. Migrazioni non applicate al riavvio

Problema: Senza automazione, le migrazioni del database dovevano essere eseguite manualmente ad ogni aggiornamento che introducesse modifiche allo schema.

Soluzione: Il comando `flask db upgrade` è stato incluso nell' `entrypoint.sh`, così viene eseguito automaticamente ad ogni avvio del container. Flask-Migrate è idempotente: se le migrazioni sono già state applicate, il comando non produce effetti collaterali.

8.4. Server homelab non raggiungibile da GitHub Actions

Problema: Il server di produzione è un homelab domestico senza IP pubblico fisso, non raggiungibile direttamente da Internet e quindi non accessibile dai runner di GitHub Actions.

Soluzione: Integrazione di **Tailscale** come VPN mesh. Il server è un nodo permanente della rete Tailscale; il runner GitHub si connette temporaneamente come nodo effimero tramite la chiave `TS_AUTHKEY`. Una volta connessi alla stessa rete virtuale, il runner può raggiungere il server via SSH come se fossero sulla stessa LAN.

9. Esperimenti eseguiti

9.1. Test dello script di validazione con post errati

Per verificare il corretto funzionamento dello script, sono stati creati post Markdown intenzionalmente invalidi e inseriti nel repository. Le casistiche testate includono:

- Post con un campo obbligatorio assente (es. `permalink` mancante)

- Post con data in formato errato (es. 20/07/2025 invece di July 20, 2025)
- Post senza il separatore -- tra metadati e contenuto
- Post completamente validi per verificare il caso di successo

In tutti i casi di errore, lo script ha restituito `exit code 1` con messaggi esplicativi, e la pipeline CI ha bloccato il job successivo come atteso.

9.2. Test della pipeline CI su Pull Request

Sono state aperte Pull Request con post invalidi verso il branch `main`. La pipeline si è attivata correttamente e ha bloccato il merge impedendo che contenuto malformato raggiungesse il branch di produzione.

9.3. Test del deploy automatico sull'homelab

Dopo aver configurato i GitHub Secrets e il nodo Tailscale sul server, è stato eseguito un push su `main` per verificare il deploy end-to-end. La sequenza di eventi osservata è stata:

1. Push su `main` → pipeline attivata
2. Job `lint-posts`: eseguito con successo
3. Job `build-and-push`: immagine Docker costruita e pubblicata su `ghcr.io`
4. Job `deploy`: runner connesso a Tailscale → SSH al server → `docker compose pull` scarica la nuova immagine → `docker compose up -d` riavvia i container
5. L'applicazione aggiornata è risultata disponibile sull'homelab

9.4. Verifica della persistenza dei dati

Riavviando i container con `docker compose down` e `docker compose up`, i dati del database (contatori di visualizzazioni) sono stati preservati grazie al volume `./postgres_data` mappato sull'host.

10. Risultati

Al termine del progetto, il workflow del maintainer si presenta come segue:

1. Il maintainer aggiunge un nuovo file Markdown in `posts/en/`
2. Crea un branch, fa commit e apre una Pull Request verso `main`
3. La pipeline CI si attiva automaticamente:
 - Se il post è invalido → pipeline fallisce, PR bloccata
 - Se il post è valido → pipeline passa, PR approvabile
4. Al merge su `main`, il deploy avviene in automatico sull'homelab

I principali risultati ottenuti sono:

- **Ambiente riproducibile:** l'intera applicazione (Flask + PostgreSQL) si avvia con un singolo `docker compose up`, senza configurazioni manuali

- **Validazione automatica:** nessun post malformato può raggiungere il branch `main` senza essere intercettato dalla pipeline
- **Immagine Docker pubblicata:** ad ogni merge su `main`, la pipeline costruisce e pubblica automaticamente l'immagine aggiornata su GHCR
- **Deploy continuo:** ogni merge su `main` produce un aggiornamento automatico dell'istanza in produzione; il server si limita a scaricare l'immagine precostruita, senza necessità di build locale
- **Gestione sicura dei segreti:** tutte le credenziali sono gestite tramite GitHub Secrets e mai esposte nel codice

11. Limitazioni e possibili miglioramenti futuri

11.1. Limitazioni attuali

- **Accessibilità limitata all'homelab:** il deployment è visibile solo ai nodi della rete Tailscale, non è accessibile pubblicamente su Internet. L'homelab è inoltre un single point of failure: se la macchina è spenta o non raggiungibile, il servizio non è disponibile.
- **Assenza di test unitari:** non sono presenti test automatici sul codice Python. La pipeline verifica solo il formato dei contenuti Markdown.
- **Assenza di orchestrazione:** l'applicazione gira su un singolo host con Docker Compose, senza nessun meccanismo di replica, failover automatico o scaling orizzontale.

11.2. Possibili miglioramenti

- **Test unitari e di integrazione:** aggiungere pytest per testare le route Flask e il comportamento del database, da eseguire nella pipeline CI.
- **Orchestrazione con Docker Swarm o Kubernetes:** per applicazioni che richiedono alta disponibilità o scaling, il passo successivo naturale sarebbe passare da Docker Compose a un orchestratore. **Docker Swarm** sarebbe la scelta più leggera e coerente con l'infrastruttura attuale. **Kubernetes** offre funzionalità più avanzate, ma introduce una complessità operativa difficilmente giustificabile per un blog a traffico ridotto.
- **Deployment su cloud pubblico (AWS):** in alternativa all'homelab, l'applicazione potrebbe essere deployata su **Amazon Web Services**. Questo eliminerebbe la dipendenza dall'homelab e garantirebbe disponibilità, backup automatici e scalabilità gestita dal provider.